

EIP-1193签名相关

背景

`web3.js` 认为用户会在本地部署以太坊节点, 私钥和网络连接状态由这个节点管理 (实际并不是这样)

在 `ethers.js` 中, 实现了读写分离, 是目前更流行, 更安全的连接以太坊方式

1. `Provider` 提供器类管理网络连接状态, 单纯使用 `Provider` 是只读的
2. `Signer` 签名者类管理密钥, 安全且灵活, `Signer` 签名者类是以太坊账户的抽象, 可用于对消息和交易进行签名
`Wallet` 钱包类继承了 `Signer` 类 (`Signer` 类是抽象类, 不能直接实例化), 并且开发者可以像包含私钥的外部拥有帐户 (EOA) 一样, 用它对交易和消息进行签名。

过去由于不同钱包提供的接口方法各不相同, dapp需要对接不同钱包的api非常麻烦, 比如要签名时, 需要针对不同钱包写多套签名过程

因此, **EIP-1193** 标准化了钱包 `Provider` API, 使得 dapp 可以操作不同的钱包, 对钱包抽象化

签名相关接口标准

```
// 返回账户地址
abstract getAddress(): Promise<string>

// 对字符串类型消息进行签名
abstract signMessage(message: Bytes | string): Promise<string>;

// 对交易类型进行签名并返回序列化后的签名结果
abstract signTransaction(transaction: Deferrable<TransactionRequest>):
Promise<string>;

// 连接指定的provider
abstract connect(provider: Provider): Signer;
```

```
// ethers.js 6.1.0
const provider = new ethers.BrowserProvider(safeAuthPack.getProvider() as
Eip1193Provider)
const signer = provider.getSigner()
```

实现参考案例

例1: `cloud-cryptographic-wallet` 是一组用于将加密库与各种云服务的密钥管理系统连接起来的软件包。这组软件包允许您执行签名并发出交易, 而无需管理私钥。

<https://github.com/odanado/cloud-cryptographic-wallet/tree/master/aws-kms-packages>

1. 实现 `ethers.js` 的 `signer` 接口:

Code

Blame

55 lines (44 loc) · 1.57 KB

Code 55% faster with GitHub Copilot

```
12 import { Adapter } from '../adapter';
13
14 // eslint-disable-next-line @typescript-eslint/no-var-requires
15 const version = require("../package.json").version;
16
17 export type KmsEthersSignerConfig = {
18   keyId: string;
19   kmsClientConfig?: KMSClientConfig;
20 };
21
22 export class KmsEthersSigner extends Signer {
23   private readonly kmsSigner: KmsSigner;
24   private readonly adapter: Adapter;
25
26   constructor(
27     private readonly config: KmsEthersSignerConfig,
28     provider?: Provider
29   ) {
30     super();
31     defineReadOnly(this, "provider", provider);
32
33     this.kmsSigner = new KmsSigner(config.keyId, config.kmsClientConfig ?? {});
34
35     this.adapter = new Adapter({ signer: this.kmsSigner, version }, provider);
36   }
37
38   async getAddress(): Promise<string> {
39     return this.adapter.getAddress();
40   }
41
42   async signMessage(message: Bytes | string): Promise<string> {
43     return this.adapter.signMessage(message);
44   }
45
46   async signTransaction(
47     deferrableTransaction: Deferrable<TransactionRequest>
48   ): Promise<string> {
49     return this.adapter.signTransaction(deferrableTransaction);
50   }
51
52   connect(provider: Provider): KmsEthersSigner {
53     return new KmsEthersSigner(this.config, provider);
54   }
55 }
```

2. 内部实际实现了向云服务器签名的过程:

```
cloud-cryptographic-wallet / aws-kms-packages / aws-kms-signer / src / signer / kms-signer.ts

Code Blame 63 lines (50 loc) · 1.69 KB Code 55% faster with GitHub Copilot

13 export class KmsSigner implements Signer {
17   public constructor(keyId: string, config?: KMSClientConfig) {
22
23   public async sign(digest: Buffer): Promise<Signature> {
24     const asn1Signature = await this._sign(digest);
25     const address = await this.getAddress();
26     const { r, s } = parseSignature(asn1Signature);
27     return Signature.fromDigest(digest, address, r, s);
28   }
29
30   public async getAddress(): Promise<Address> {
31     const asn1PublicKey = await this.getPublicKey();
32
33     const publicKey = parsePublicKey(asn1PublicKey);
34     return Address.fromPublicKey(publicKey);
35   }
36
37   public async getPublicKey(): Promise<Buffer> {
38     const command = new GetPublicKeyCommand({ KeyId: this.keyId });
39
40     const response = await this.client.send(command);
41     if (!response.PublicKey) {
42       throw new TypeError("PublicKey is undefined");
43     }
44
45     return Buffer.from(response.PublicKey);
46   }
47
48   private async _sign(digest: Buffer) {
49     const command = new SignCommand({
50       KeyId: this.keyId,
51       Message: digest,
52       MessageType: "DIGEST",
53       SigningAlgorithm: "ECDSA_SHA_256",
54     });
55     const response = await this.client.send(command);
56
57     if (!response.Signature) {
58       throw new TypeError("Signature is undefined");
59     }
60
61     return Buffer.from(response.Signature);
62   }
63 }
```

例2: Web3Auth 提供了社交账户技术, safe 钱包和 web3Auth 合作提供的JavaScript SDK 中, 对 Web3Auth的身份认证MPC服务进行了集成

<https://github.com/safe-global/safe-core-sdk/blob/357d6b2cbfa4ba04e09edb7fe313e1ac00bd03eb/packages/protocol-kit/src/Safe.ts>

1. protocol-kit包中实现了provider类和sign类的接口

```
safe-core-sdk / packages / protocol-kit / src / SafeProvider.ts

Code Blame 280 lines (239 loc) · 7.67 KB

38 class SafeProvider {
223   async getAddress(): Promise<string | undefined> {
228
229   async signMessage(message: string): Promise<string> {
230     const signer = await this.getExternalSigner()
231
232     if (!signer) {
233       throw new Error('SafeProvider must be initialized with a signer to use this method')
234     }
235     const messageArray = ethers.getBytes(message)
236
237     return signer.signMessage(messageArray)
238   }
239
240   async signTypedData(safeEIP712Args: SafeEIP712Args): Promise<string> {
241     const signer = await this.getExternalSigner()
242
243     if (!signer) {
244       throw new Error('SafeProvider must be initialized with a signer to use this method')
245     }
246
247     if (isTypedDataSigner(signer)) {
248       const typedData = generateTypedData(safeEIP712Args)
249       const signature = await signer.signTypedData(
250         typedData.domain,
251         typedData.primaryType === 'SafeMessage'
252           ? { SafeMessage: (typedData as EIP712TypedDataMessage).types.SafeMessage }
253           : { SafeTx: (typedData as EIP712TypedDataTx).types.SafeTx },
254         typedData.message
255       )
256       return signature
257     }
258
259     throw new Error('The current signer does not implement EIP-712 to sign typed data')
260   }
261
262   async estimateGas(transaction: SafeProviderTransaction): Promise<string> {
263     return (await this.#externalProvider.estimateGas(transaction)).toString()
264   }
}
```

2. 具体实现中实现了社交身份认证的多签过程校验

Code Blame 1539 lines (1389 loc) · 52.2 KB

```

79     class Safe {
784         * @returns The signed Safe transaction
785         * @throws "Transactions can only be signed by Safe owners"
786         */
787         async signTransaction(
788             safeTransaction: SafeTransaction | SafeMultisigTransactionResponse,
789             signingMethod: SigningMethodType = SigningMethod.ETH_SIGN_TYPED_DATA_V4,
790             preimageSafeAddress?: string
791         ): Promise<SafeTransaction> {
792             const transaction = isSafeMultisigTransactionResponse(safeTransaction)
793                 ? await this.toSafeTransactionType(safeTransaction)
794                 : safeTransaction
795
796             const owners = await this.getOwners()
797             const signerAddress = await this.#safeProvider.getSignerAddress()
798
799             if (!signerAddress) {
800                 throw new Error('SafeProvider must be initialized with a signer to use this method')
801             }
802
803             const addressIsOwner = owners.some(
804                 (owner: string) => signerAddress && sameString(owner, signerAddress)
805             )
806             if (!addressIsOwner) {
807                 throw new Error('Transactions can only be signed by Safe owners')
808             }
809
810             const safeVersion = await this.getContractVersion()
811             if (
812                 signingMethod === SigningMethod.SAFE_SIGNATURE &&
813                 senderSatisfies(safeVersion, EQ_OR_GT_1_3_0) &&
814                 !preimageSafeAddress
815             ) {
816                 throw new Error('The parent Safe account address is mandatory for contract signature')
817             }
818
819             let signature: SafeSignature
820
821             if (signingMethod === SigningMethod.ETH_SIGN_TYPED_DATA_V4) {
822                 signature = await this.signTypedData(transaction, 'v4')
823             } else if (signingMethod === SigningMethod.ETH_SIGN_TYPED_DATA_V3) {
824                 signature = await this.signTypedData(transaction, 'v3')
825             } else if (signingMethod === SigningMethod.ETH_SIGN_TYPED_DATA) {
826                 signature = await this.signTypedData(transaction, undefined)
827             } else {
828                 const safeVersion = await this.getContractVersion()
829                 const chainId = await this.getChainId()
830                 if (!hasSafeFeature(SAFE_FEATURES.ETH_SIGN, safeVersion)) {
831                     throw new Error('eth_sign is only supported by Safes >= v1.1.0')
832                 }
833
834                 let txHash: string
835
836                 // IMPORTANT: because the safe uses the old EIP-1271 interface which uses 'bytes' in
837                 // we need to use the pre-image of the transaction hash to calculate the message has

```